

FastAPI CRUD Q1

Build a basic Student Management System using FastAPI.

Store student records temporarily using a Python list.

Requirements

Each student should contain:

- `id`
- `name`
- `age`
- `course`
- `marks`

Use proper HTTP methods and return JSON responses.

Questions

1. Create FastAPI Application

Create a basic FastAPI application and run the server successfully.

2. Create API – Add Student

Create an API endpoint to add a new student.

Endpoint

`POST /students`

Input Example

```
{  
  "name": "Rahul",  
  "age": 21,  
  "course": "Python",  
  "marks": 85  
}
```

Expected Output

```
{  
  "message": "Student added successfully"  
}
```

3. Create API – Get All Students

Create an API endpoint to fetch all student records.

Endpoint

GET /students

4. Create API – Get Student By ID

Create an API endpoint to fetch a student using student ID.

Endpoint

GET /students/{id}

5. Create API – Update Student

Create an API endpoint to update student details.

Endpoint

PUT /students/{id}

Task

Update:

- name
 - age
 - course
 - marks
-

6. Create API – Delete Student

Create an API endpoint to delete a student record.

Endpoint

`DELETE /students/{id}`

Expected Output

```
{
  "message": "Student deleted successfully"
}
```

Additional Questions

1. What is CRUD in FastAPI?
2. Why do we use the POST method for inserting data?
3. Difference between `PUT` and `PATCH`
4. What is the path parameter?
5. What is the request body in FastAPI?
6. Why do we use Pydantic models?
7. What status code is returned for successful deletion?
8. How can we check whether a student exists before updating?
9. What happens if duplicate IDs are inserted?
10. How does FastAPI automatically generate Swagger UI?